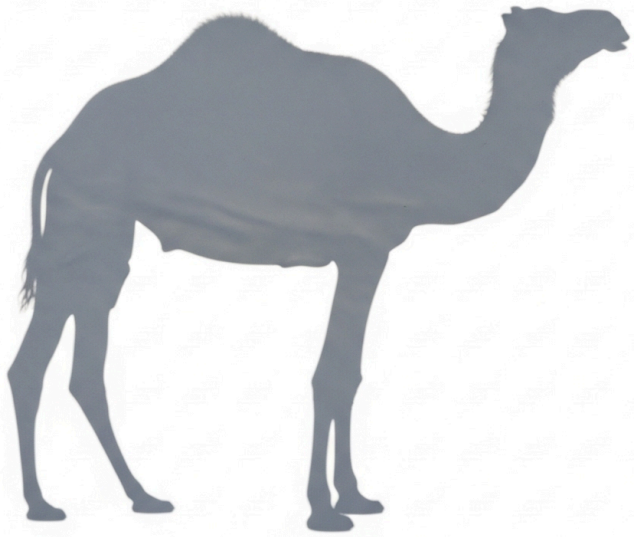
QSWAP

Security Assessment and Code Audit



Mundus

Jan 18th 2026

Rev 2

INTRODUCTION

QSwap is a Uniswap V2-style Automated Market Maker (AMM) DEX for the Qubic blockchain. It enables users to create liquidity pools for QU/Asset pairs, provide liquidity, and swap between QU and various assets.

The contract implements constant product market making ($x \cdot y = k$) with a 0.3% swap fee distributed among

- Shareholders (27%)
- QX (5%)
- Invest & Rewards (3%)
- Burn (1%)
- Liquidity providers (64%)

AUDIT PROCESS

Following considerations have been tested and reviewed by Mundus team:

- Reviewing google test files that are written by SC developer.
- Trying several attack vectors to exploit the SC.
- Ensuring contract logic meets the specifications.
- Thorough line-by-line manual review.

RESULTS

We identified a range of security issues in our assessment, from medium to minor. We recommend fixing them to meet security standards and best practices.

OVERVIEW

1. Project name: **QSWAP**
2. Platform: **QUBIC SC system**
3. Is new SC deployment: **NO**
4. Is upgraded SC: **YES**
5. Project size: Large
6. List of github commits to review:
    a. 57d24af49c3680b2f54e39231e8ed7c721465929
    b. 115ca4744e037b992f89776984427df09d8a84af

RESULTS SUMMARY

Total Issues	10
Critical	0
Major	0
Medium	1
Minor	9

AUDIT SCOPE

Filename	SHA	Latest commit
Qswap.h	57d24af49c3680b2f54e39231e8ed7c721465929	epoch -> tick
contract_qswap.cpp (gtest)	115ca4744e037b992f89776984427df09d8a84af	fix unit tests

STATUS OF FINDINGS IN REV 1

ID	Title	Category	Severity	Status
QSWAP-01	TransferShareManagementRights Fee Handling Issues	Logic	Medium	Fixed
QSWAP-02	CreatePool Only Allows Asset Issuer Despite Different Input	Logic	Minor	Not Fixed
QSWAP-03	No Fee Rate Validation	Logic	Minor	Fixed
QSWAP-04	Potential Reserve Underflow in Pool State Updates	Logic	Minor	Fixed
QSWAP-05	Redundant/Duplicate Code in SwapQuForExactAsset	Logic	Minor	Fixed
QSWAP-06	Typo in Comments	Code Convention	Minor	Fixed
QSWAP-07	Failed unit test: QuoteTest	Logic	Minor	Fixed
QSWAP-08	Failed unit test: SwapAssetForExactQu	Logic	Minor	Fixed
QSWAP-9	Slow traversing for the array `state.mPoolBasicStates`	Logic	Minor	Acknowledged - Not Fixed
QSWAP-10	Inefficient Pool Slot Recycling	Logic	Minor	Acknowledged - Not Fixed

SUMMARY OF FINDINGS

ID	Title	Category	Severity
QSWAP-11	Wasteful Cross-Contract Calls for Static Fees	Logic	Minor
QSWAP-12	Inefficient TransferShareManagementRights Logic	Logic	Minor
QSWAP-13	Slow Square Root Algorithm	Logic	Minor

FINDINGS

QSWAP-11 Wasteful Cross-Contract Calls for Static Fees

- **File(s) affected:** Qswap.h:2189, 514, 869, 942
- **Description:** The contract calls the external contract `QX` to fetch fee settings in every execution of critical procedures and functions (Fees, IssueAsset, CreatePool, TransferShareOwnershipAndPossession) using `CALL_OTHER_CONTRACT_FUNCTION`.
- **Impact:** Inter-contract calls (`CALL_OTHER_CONTRACT_FUNCTION`) are expensive operations. Fee settings (like `assetIssuanceFee`) change very rarely, making this redundant overhead for every user transaction.
- **Recommendation:** Cache the QX fees in `QSWAP`'s state variables (e.g., `state.cachedIssuanceFee`). Update the new fees at the beginning of each epoch.

QSWAP-12 Inefficient `TransferShareManagementRights` Logic

- **File(s) affected:** Qswap.h:2267-2336
- **Description:** The procedure attempts to release shares with a 0 fee first to "probe" the required fee, fails, and then calls it again with the correct fee.
- **Impact:** Calling a system procedure twice doubles the overhead.
- **Recommendation:** Call `qpi.releaseShares` once, passing the full `qpi.invocationReward()` (or `locals.reward`) as the `offeredTransferFee`. The system will automatically deduct only the required fee and return the success status.

QSWAP-13 Slow Square Root Algorithm

- **File(s) affected:** Qswap.h:354-375
- **Description:** The `sqrt` function uses a division-heavy iterative method inside `AddLiquidity`.
- **Impact:** `div` (division) is CPU-intensive. For large liquidity values, this loop can run many times, consuming significant ticks.
- **Recommendation:** Replace with a bitwise integer square root algorithm, which relies on bit-shifts and additions and is significantly faster for the `uint128` data type involved here.